

Пояснительная записка к проекту

по курсу: Структуры и алгоритмы обработки данных

по теме: Алгоритм поиска иголки в стоге сена методом последовательного перебора

организация: Филиал МГУ имени М.В. Ломоносова в городе Дубне

направление: Методы и технологии обработки данных в гетерогенных вычислительных средах

выполнил: студент 5-го курса Чеснокова Мария Алексеевна

дата: 4 июня 2026 г.

Введение

В проекте рассматривается задача быстрого приближённого вычисления расстояний в разреженных графах. Пусть задан взвешенный неориентированный граф $G = (V, E)$, где $|V| = n$ - количество вершин графа, $|E| = m$ - количество рёбер графа, чьи веса неотрицательны. Требуется построить структуру данных, которая после предварительной обработки графа позволяет быстро отвечать на запросы расстояний между парами вершин.

Целью работы является разработка и исследование структуры данных для приближённого вычисления расстояний с контролируемым компромиссом между точностью, временем ответа и объёмом памяти. В качестве точного эталона используется алгоритм Дейкстры [1]. В качестве приближённых методов реализованы landmark-оракулы и реализация оракула Thorup-Zwick [2]. Для обоих подходов реализованы базовые и модифицированные версии.

Описание реализованных алгоритмов и обоснование выбора

В проекте реализованы несколько алгоритмов и структур данных, предназначенных для точного и приближённого вычисления расстояний в разреженных графах. В качестве точного базового алгоритма используется алгоритм Дейкстры, а для ускорения запросов реализованы landmark-оракулы и оракулы Thorup-Zwick.

Список смежности

Для хранения графа используется список смежности. Каждой вершине соответствует список её соседей, а каждое ребро хранит номер конечной вершины и вес. Для неориентированного графа ребро добавляется в оба направления.

Данная структура была выбрана потому, что рассматриваемые графы являются разреженными. Для таких графов матрица смежности была бы неэффективной, так как требовала бы $O(n^2)$ памяти независимо от числа рёбер. Список смежности использует память пропорционально реальному количеству рёбер.

Пространственная сложность хранения графа: $O(n+m)$.

Добавление одного ребра без учёта дополнительных проверок выполняется за: $O(1)$.

Обход всех соседей вершины v выполняется за: $O(d(v))$, где $d(v)$ — степень вершины.

Список смежности используется всеми реализованными алгоритмами: алгоритмом Дейкстры, landmark-оракулами, Thorup-Zwick oracle и генератором случайных графов.

Алгоритм Дейкстры

Алгоритм Дейкстры используется как точный эталонный алгоритм для вычисления кратчайших расстояний. Он был выбран по нескольким причинам. Во-первых, графы в проекте имеют неотрицательные веса рёбер. Во-вторых, алгоритм хорошо подходит для разреженных графов при реализации через очередь с приоритетом. В-третьих, он позволяет получать точное расстояние, с которым можно сравнивать приближённые ответы оракулов.

В реализации для каждой вершины хранится текущее минимальное найденное расстояние от стартовой вершины. На каждом шаге из очереди с приоритетом извлекается вершина с минимальным текущим расстоянием, после чего выполняется релаксация всех исходящих из

неё рёбер. Если найден более короткий путь до соседней вершины, расстояние обновляется, а новая пара "расстояние–вершина" добавляется в очередь.

Результатом одного запуска алгоритма из вершины s является массив $dist_s[v]$ — точное кратчайшее расстояние от вершины s до вершины v .

В проекте алгоритм Дейкстры используется в двух целях. Первая цель — получение точного ответа для каждого экспериментального запроса. Вторая цель — предварительное вычисление расстояний при построении оракулов.

Временная сложность одного запуска алгоритма Дейкстры при использовании очереди с приоритетом: $O((n + m) \log n)$.

Landmark-оракул

Landmark-оракул является простым приближённым методом для быстрого ответа на запросы расстояний. Он был выбран как базовый практический метод. Основная его идея состоит в выборе L специальных вершин, называемых landmark-вершинами. Для каждой такой вершины заранее запускается алгоритм Дейкстры и сохраняются расстояния до всех остальных вершин графа. После этого запрос между вершинами u и v обрабатывается не запуском Дейкстры, а перебором всех landmark-вершин.

Приближённое расстояние вычисляется по формуле:

$$\tilde{d}(u, v) = \min_{1 \leq i \leq L} (dist_{l_i}[u] + dist_{l_i}[v]),$$

где l_i — очередная landmark-вершина.

Такой ответ соответствует длине пути из u в v через одну из landmark-вершин. Поэтому приближённое расстояние не должно быть меньше точного кратчайшего расстояния, но может быть больше него.

В проекте реализованы две версии landmark-оракула.

Оптимизированная версия выбирает вершины с наибольшей степенью. Такой выбор основан на предположении, что вершины с большим числом связей лучше покрывают структуру графа и чаще дают более короткий путь через landmark-вершину.

Временная сложность построения landmark-оракула: $O(L(n + m) \log n)$, так как необходимо выполнить L запусков алгоритма Дейкстры.

Пространственная сложность: $O(Ln)$, поскольку для каждой из L landmark-вершин хранится расстояние до каждой из n вершин графа.

Временная сложность одного запроса: $O(L)$, так как нужно перебрать все landmark-вершины.

Для оптимизированной версии дополнительно требуется сортировка вершин по степени: $O(n \log n)$.

Оракул Thorup–Zwick

Оракул Thorup–Zwick был выбран как известная теоретически обоснованная схема приближённых оракулов расстояний. В отличие от простого landmark-подхода, этот метод имеет параметр k , который связан с теоретической гарантией качества приближения. Для классической схемы stretch равен: $2k - 1$.

Это означает, что для точного расстояния $d(u, v)$ и приближённого расстояния $\tilde{d}(u, v)$ выполняется оценка:

$$d(u, v) \leq \tilde{d}(u, v) \leq (2k - 1)d(u, v).$$

В классическом подходе строится последовательность вложенных множеств:

$$A_0 \supseteq A_1 \supseteq \dots \supseteq A_k,$$

где: $A_0 = V$, $A_k = \emptyset$.

Для каждой вершины v на каждом уровне определяется ближайшая вершина из множества A_i , называемая pivot-вершиной: $p_i(v)$.

Кроме того, для каждой вершины строится множество $B(v)$, называемое bunch. Оно хранит часть близких и структурно важных вершин, которые используются при ответе на запрос.

Запрос между вершинами u и v выполняется путём поиска такой вершины w , которая является pivot-вершиной для одной из вершин и содержится в bunch другой вершины. После этого ответ вычисляется как:

$$\tilde{d}(u, v) = d(u, w) + d(w, v).$$

Для построения сначала вычисляются кратчайшие расстояния между всеми парами вершин с помощью запусков алгоритма Дейкстры из каждой вершины. Затем эти расстояния используются для построения pivot-вершин и bunch-множеств.

Временная сложность этапа предварительного вычисления всех расстояний: $O(n(n+m) \log n)$.

Пиковая пространственная сложность на этапе построения: $O(n^2)$, так как временно хранится матрица кратчайших расстояний между вершинами.

Пространственная сложность итоговой структуры данных: $O(kn + B)$, где B — суммарный размер всех bunch-множеств.

Временная сложность одного запроса: $O(k)$, поскольку алгоритм последовательно рассматривает уровни и выполняет поиск нужной вершины в bunch-множестве.

В проекте реализованы две версии Thorup–Zwick oracle.

Простая версия строит уровни случайным образом. Вероятность попадания вершины на следующий уровень задаётся как: $p = n^{-1/k}$.

Оптимизированная версия является модификацией базовой схемы. В ней вероятность выбора вершины зависит от её степени:

$$p_v = \min \left(1; p \cdot \frac{d(v)}{\bar{d}} \right),$$

где $\bar{d}$ — средняя степень вершины в графе. Эта модификация была выбрана потому, что вершины с высокой степенью часто являются более важными для структуры графа и могут улучшать качество приближённого ответа. При этом асимптотическая сложность построения и запроса остаётся такой же, как у базовой версии.

Генератор случайных графов

Генератор случайных графов используется для создания синтетических разреженных графов разных размеров и плотности. Он был реализован для того, чтобы можно было проводить эксперименты не только на реальных данных, но и на контролируемых графах с заранее заданным числом вершин и рёбер.

Генерация выполняется в два этапа. Сначала создаётся цепочка: $0 - 1 - 2 - \dots - (n - 1)$, которая гарантирует связность графа. Затем добавляются случайные рёбра до достижения заданного числа рёбер m . Петли и повторяющиеся рёбра не добавляются. Вес каждого ребра выбирается случайно из заданного диапазона.

Такой способ был выбран потому, что он гарантирует связность графа и одновременно позволяет получать разреженные графы с различным числом вершин и рёбер.

Временная сложность генерации: $O(m \log m)$.

Пространственная сложность: $O(n + m)$.

Сгенерированные графы сохраняются в текстовые файлы и затем используются в экспериментальной части так же, как реальные графы.

Заключение

В ходе выполнения работы были реализованы и исследованы структуры данных для приближённого вычисления расстояний в разреженных графах. В качестве точного метода использовался алгоритм Дейкстры, позволяющий находить кратчайшие расстояния в графах с

неотрицательными весами рёбер. С его помощью проверялась точность приближённых ответов, полученных оракулами расстояний.

Для ускорения ответов на запросы были реализованы landmark-оракулы и версия оракула Thorup–Zwick. Для landmark метода были реализованы две стратегии выбора landmark-вершин: базовая стратегия выбора по индексу и оптимизированная стратегия выбора вершин с наибольшей степенью. В качестве модификации для Thorup–Zwick oracle была реализована стратегия degree-biased sampling, в которой вероятность попадания вершины на следующий уровень зависит от её степени.

В экспериментальной части методы сравнивались по времени построения, времени ответа на запрос, объёму используемой памяти и ошибке относительно точного расстояния.

Список литературы

1. Dijkstra E. W. A note on two problems in connexion with graphs // Numerische Mathematik. 1959. Vol. 1. P. 269–271. DOI: [10.1007/BF01386390](https://doi.org/10.1007/BF01386390).
2. Thorup M., Zwick U. Approximate distance oracles // Journal of the ACM. 2005. Vol. 52, No. 1. P. 1–24. DOI: [10.1145/1044731.1044732](https://doi.org/10.1145/1044731.1044732).
3. Leskovec J., Krevl A. SNAP Datasets: Stanford Large Network Dataset Collection. 2014. URL: <https://snap.stanford.edu/data/>.